P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Introdução Web APIs

Bruno Oliveira (bmo@estg.ipp.pt)

PEI Processamento Estruturado de Informação

Arquitetura Cliente-Servidor

- A comunicação entre um browser e um site utiliza a arquitetura Cliente-Servidor.
- Os clientes (por exemplo web browser) realizam pedidos de forma a requisitar recursos disponibilizados por servidores web;
- Esta troca de mensagens é efetuada de acordo com o protocolo HTTP, que é a base de qualquer troca de dados na web.

Arquitetura Cliente-Servidor

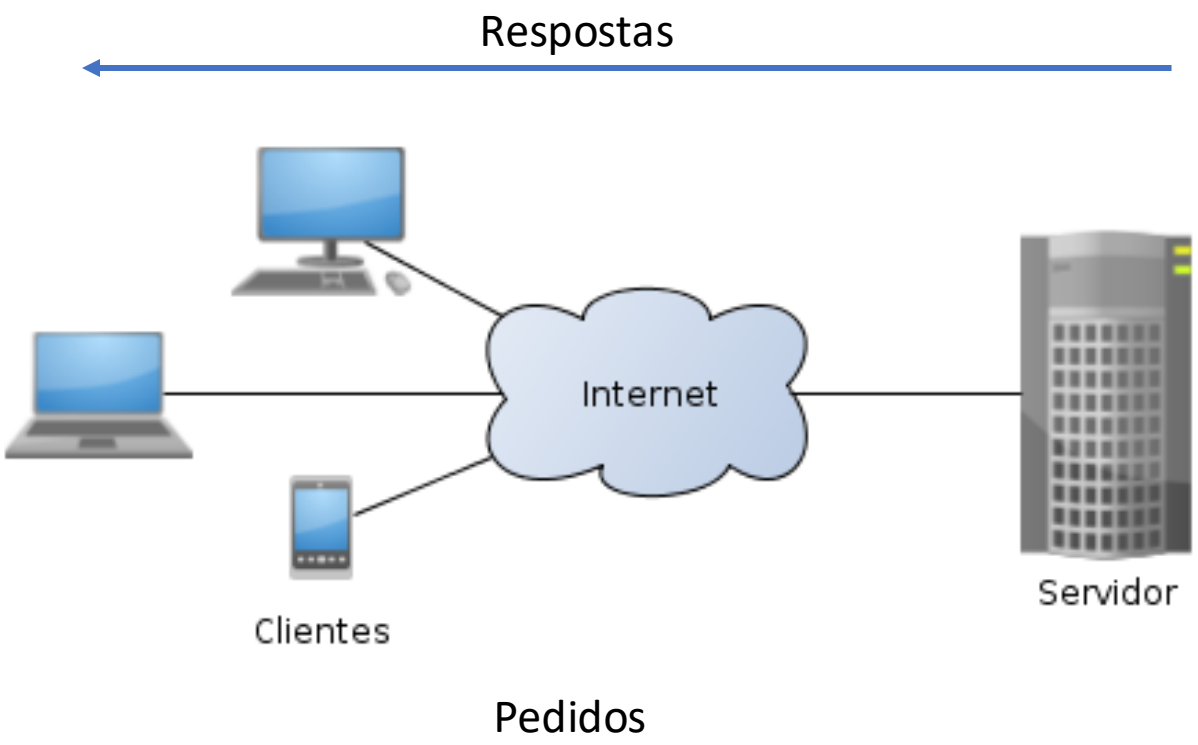


Imagem Por Tiago de Jesus Neves - Obra do próprio, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=26681739

Arquitetura Cliente-Servidor

- Os dados são trocados entre computadores através de "packets" que descrevem o formato com que os dados são enviados;
- O browser (por exemplo) é responsável por reunir todos os pacotes num website e apresentar o seu conteúdo.

O que é o pedido HTTP?

- O mensagem do pedido contém:
  - Request-line: identifica o Request method, o recurso a solicitar e a versão do HTTP;
  - Request method: GET, HEAD, POST, PUT, DELETE, CONNECT, OPTIONS e TRACE, indicando o método de enviado da informação para o servidor;

O que é o pedido HTTP?

- O mensagem do pedido contém:
  - Request Header: contém informação adicional sobre o pedido, como por exemplo: preferências de idioma ou o tipo de cliente utilizado;
  - Request Body: contém conteúdo que será enviado para o servidor, utilizando por exemplo XML ou JSON.

O que é o pedido HTTP?

Linha de requisição

Linhas de cabeçalho

Corpo

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Content-Type: text/xml; charset=utf-8
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

<?xml version="1.0" encoding="utf-8"?>
<string xmlns="http://clearforest.com/">string</string>
```

Exemplo modificado de: https://www.tutorialspoint.com/http/http_requests.htm

Tipos de resposta HTTP

- O mensagem da resposta contém:
  - Message Status-Line: consiste na identificação da versão do protocolo HTTP, um código de estado e uma identificação textual do estado;
  - Status Code: 3 dígitos to tipo inteiro em que o primeiro dígito identifica a categoria do estado associado à resposta: 1XX – informação, 2XX- Sucesso, 3XX- redireccionamento, 4XX – Erro por parte do cliente, 5XX – Erro por parte do servidor;
  - Response Header Fields: informação adicional sobre a resposta relacionada com o servidor ou com pedidos futuros;

O que é a resposta HTTP?

Linha de estado

Linhas de cabeçalho

Corpo

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: application/json
Connection: Closed

{"id":1,"content":"Hello, World!"}
```

MIME types: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Basico_sobre_HTTP/MIME_types

Exemplo modificado de: https://www.tutorialspoint.com/http/http_requests.htm

Serviços Web

- Serviços são componentes Web que podem ser publicados, encontrados e utilizados através da internet;
- Representam a comunicação entre aplicações através da internet;
- Assumem um papel crucial na troca de dados entre sistemas, principalmente entre sistemas heterógenos onde a complexidade de comunicação é mais significativa;

Serviços SOAP

- SOAP (Simple Object Access Protocol) é um protocolo de comunicação que define o formato para a troca de mensagens com um serviço web;
- Uma mensagem SOAP é um documento XML com os seguintes elementos:
  - Um elemento: Envelope que identifica o documento XML como uma mensagem SOAP;
  - Um elemento: Header que contém informação do cabeçalho;
  - Um elemento: Body que contém informação sobre as mensagens;
  - Um elemento: Fault que contém informação de erro e estado;

Serviços SOAP

SOAP Envelope element

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soap:Body>
        <Guru99WebService xmlns="http://tempuri.org/">
          <TutorialID>int</TutorialID>
        </Guru99WebService>
  </soap:Body>
</soap:Envelope>
```

SOAP Body element

Name of the web service

Parameter required by the web service

Serviços REST

- REST representa um estilo de arquitetura que tem por base o protocolo HTTP;
- Numa arquitetura REST são disponibilizados recursos que é acedido através de uma interface comum baseado em métodos HTTP standard;
- É utilizado um servidor que disponibiliza o acesso aos recursos, enquanto que o cliente REST pode aceder e modificar os recursos REST;

Serviços REST

- Um pedido HTTP é equivalente a uma invocação de um método (operação) de um recurso disponível no servidor;
- O método HTTP é utilizado para determinar a operação a realizar sobre um determinado recurso;
- Cada recurso é identificado por URIs (Uniform Resource Identifier) e deverá suportar as operações HTTP;

Serviços REST

- Os parâmetros podem ser enviados no URL e/ou no corpo do pedido;
- Os tipos de dados utilizados no pedido e na resposta devem ser acordados entre o servidor e o(s) cliente(s);
- JSON e XML estão entre os tipos mais utilizados.

SOAP ou REST?

- Vantagens SOAP:
  - SOAP é mais adequado para sistemas mais complexos e que requeiram uma segurança mais robusta.
  - Além disso, também fornece um sistema de mensagens de falha integrado.
- Vantagens REST:
  - REST é mais simples, flexível e rápido.
  - Além disso, permite uma maior diversidade de formatos de dados e pode recorrer a um maior número de métodos HTTP.

REST Web API

- Uma (web) API representa uma interface web para o software;
- Pode ser visto como um contrato entre um fornecedor de dados e um consumidor estabelecendo o conteúdo exigido pelo consumidor (o pedido) e o conteúdo exigido pelo fornecedor (a resposta).
- Por exemplo, o design da API para um serviço de meteorologia pode exigir que o utilizador forneça um código postal e que o fornecedor responda com as devidas temperaturas.

REST Web API

- Com REST, os recursos são solicitados como se tratassem de uma caixa negra em que os pormenores de implementação não são conhecidos;
- REST é stateless, i.e., nenhum estado é mantido entre execuções do serviço.

Exemplo de Web API

- Existem muitas APIs públicas que podem ser utilizadas e integradas em aplicações. Exemplos: https://github.com/public-apis/public-apis
- A AmiiboAPI é uma RESTful API que permite consultar informação acerca de amiibos:

$$\underbrace{\text{https://www.amiiboapi.com}}_{\text{endereço}}\underbrace{\text{/api/amiibo/}}_{\textit{recurso}}\underbrace{\text{?character=zelda}}_{\text{parâmetros}}$$

REST – API Call

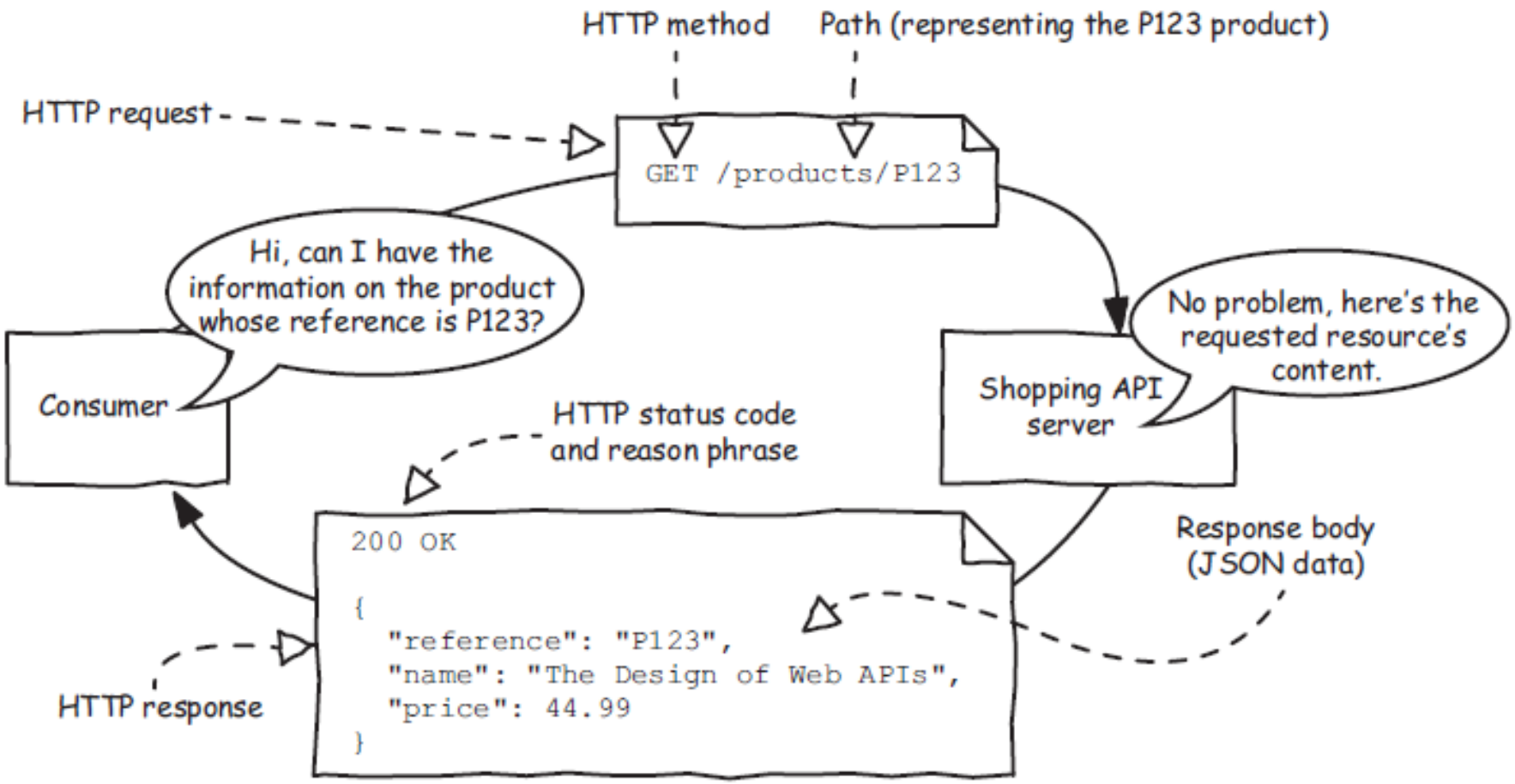


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Princípios básicos de uma REST API

- Uma API REST depende totalmente do protocolo HTTP;

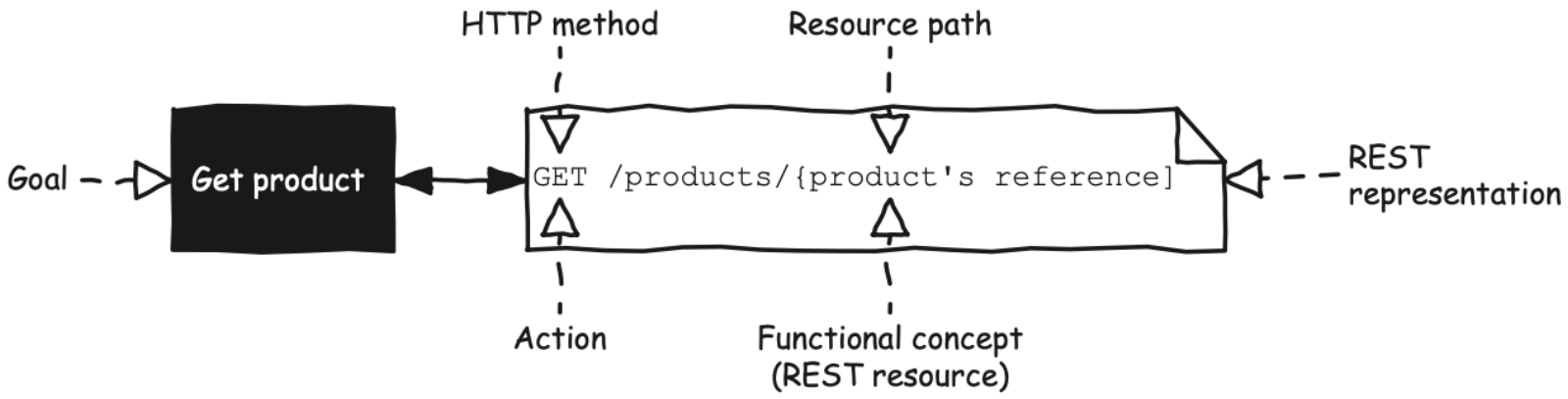


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Representar ações com HTTP

- Quando adicionamos um produto a um catálogo identificado por /products, criamos um resource de produto:

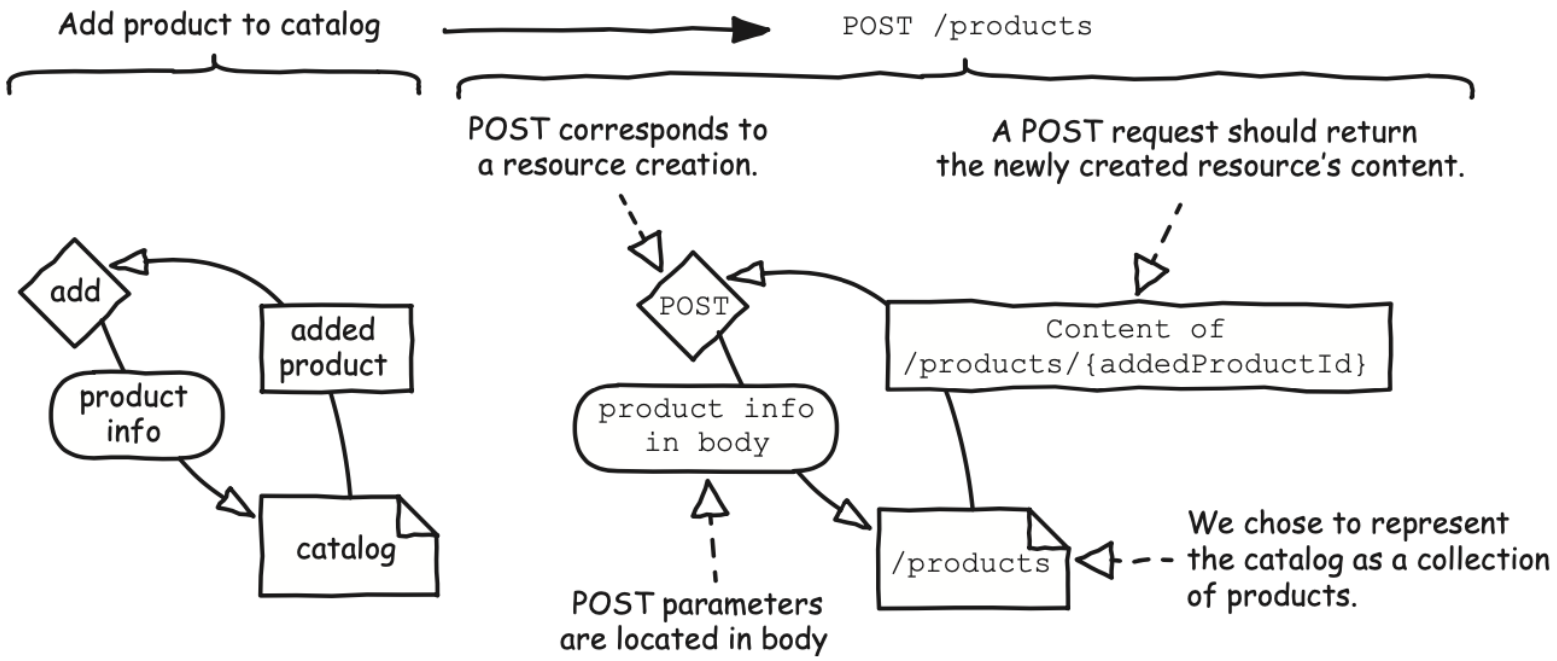


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Representar ações com HTTP

- A representação HTTP da pesquisa de produtos no catálogo usando uma consulta livre é GET.

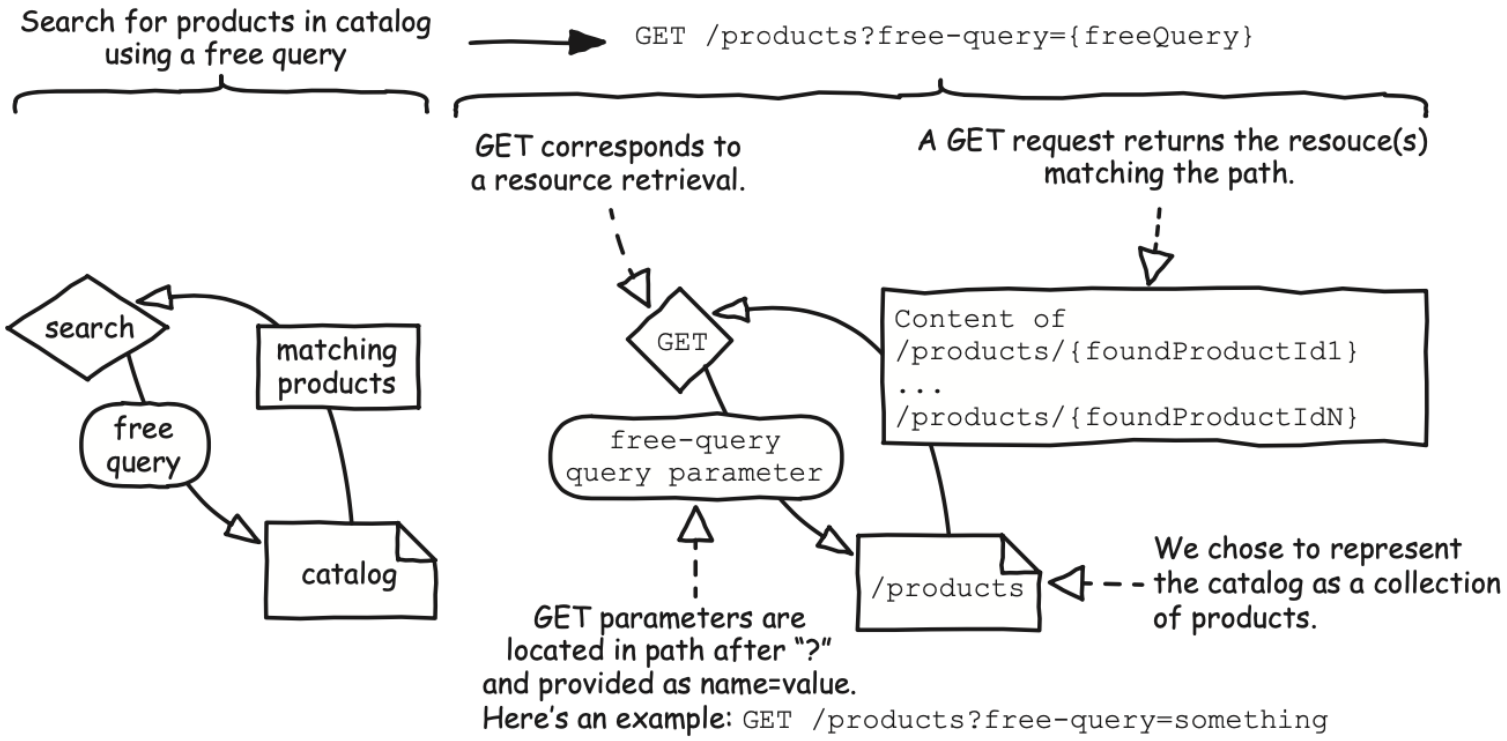


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Representar ações com HTTP

- A representação HTTP para a “eliminação” de um produto no catálogo é DELETE.

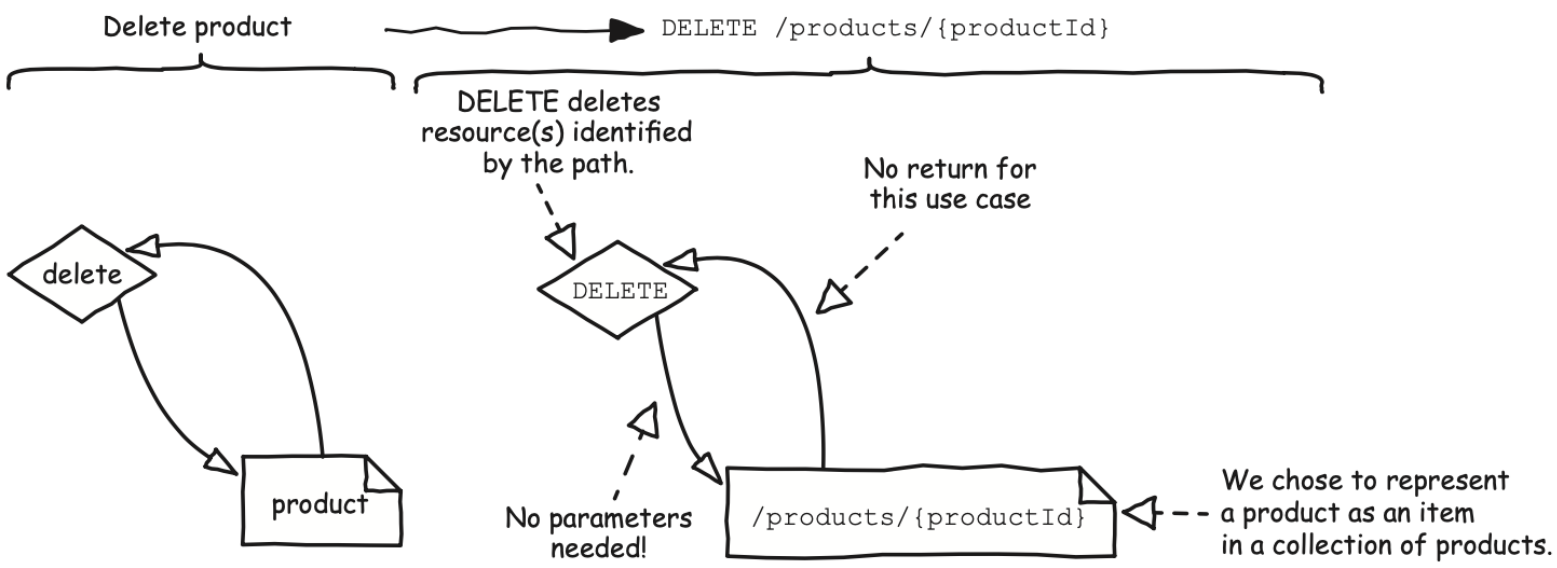


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Representar ações com HTTP

- A representação HTTP para a atualização de um produto no catálogo é PATCH.

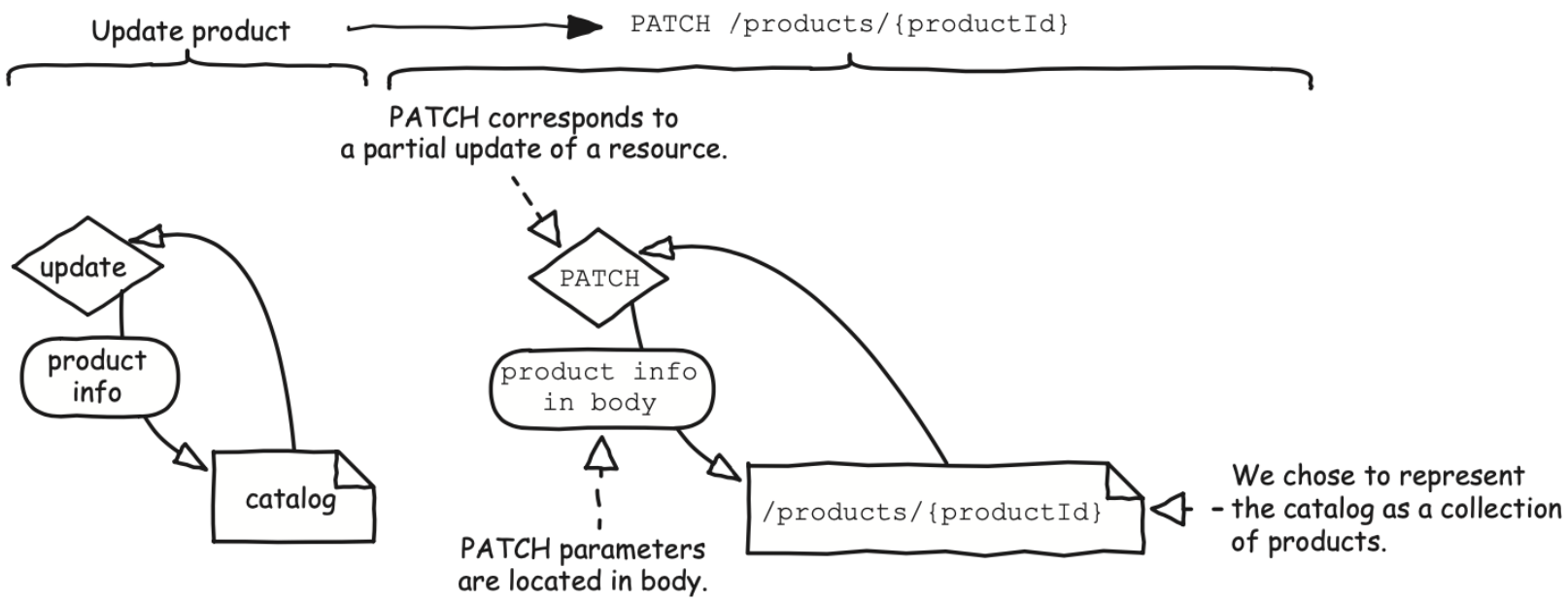


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Representar ações com HTTP

- O método PUT pode ser usado para substituir totalmente um recurso existente ou para criar um inexistente e fornecer seu identificador

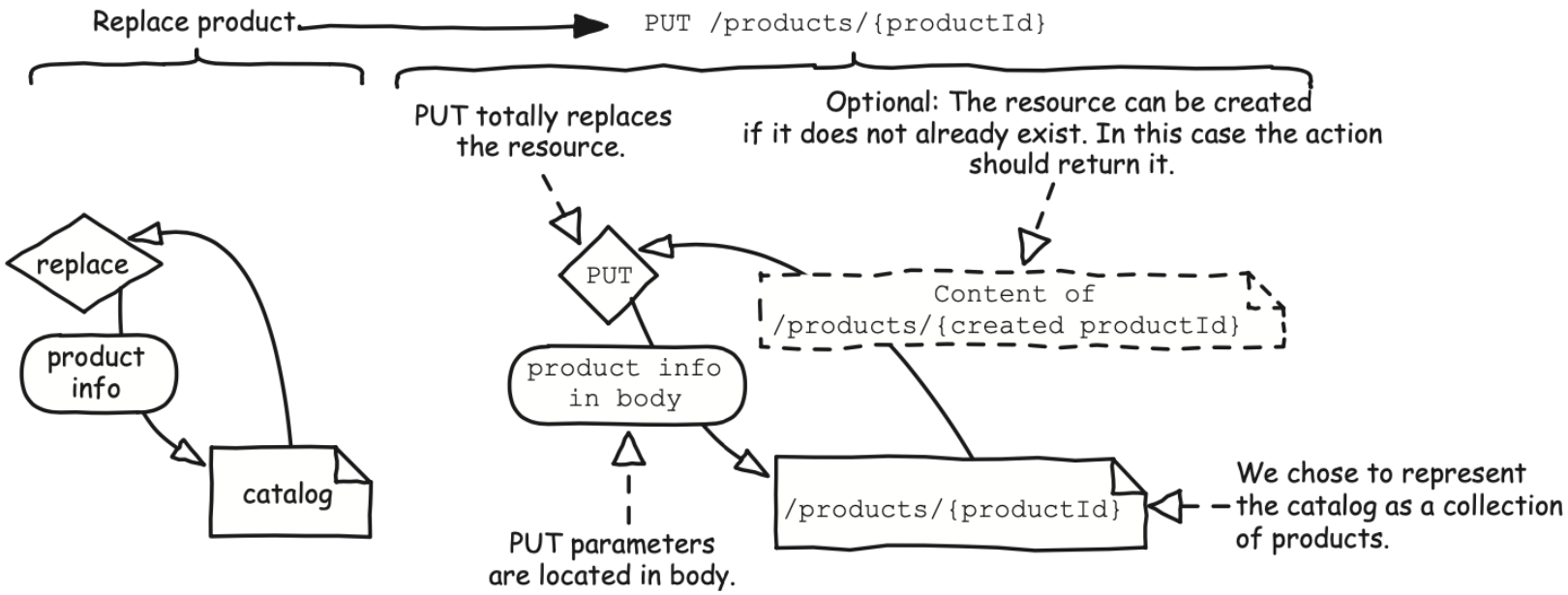


Imagem retirada do livro:Arnaud Lauret - The Design of Web APIs

Representar ações com HTTP

- Estas ações são realizadas do ponto de vista do consumidor!

Método HTTP	Ação
POST / PUT se for utilizado para criar recursos	Criar um cliente, adicionar uma refeição a um menu, enviar uma mensagem para o atendimento ao cliente, assinar um serviço, abrir conta bancária, upload de uma foto, (…)
GET	Ler um cliente, procurar um restaurante francês, encontrar novos amigos, recuperar contas abertas nos últimos 3 meses, download um contrato assinado, filtrar livros mais vendidos, selecionar fotos, listar amigos, (…)
PATCH/PUT	Atualizar um cliente, substituir mercadorias num pedido, trocar de assento de avião, editar o método de entrega de um pedido, alterar a moeda de um pedido, modificar um limite de cartão de débito, bloquear temporariamente um cartão de crédito, (…)
DELETE	Excluir um cliente, cancelar um pedido, fechar um caso, encerrar um processo, interromper um cronômetro, (…)

Implementação de uma REST API com Nodejs e Express

- Node.js é um runtime JavaScript de código aberto que permite executar JavaScript no lado do servidor.
- O Express.js é uma framework web minimalista para Node.js, utilizado para criar aplicações e APIs web.
- O Node.js fornece a base para executar JavaScript no servidor.
- O Express.js adiciona abstrações para simplificar o desenvolvimento.
  - Node.js gere ligações e executa o código.
  - O Express faz gestão das rotas, pedidos e middlewares utilizados.

Implementação de uma REST API com Nodejs e Express

- Realizar o download e instalar do Node.js (https://nodejs.org/en)
- Após a instalação do NodeJs, é necessário instalar o pacote (linha de comandos): npm install express;
- Utilizando um editor de texto, numa pasta específica criar um ficheiro app.js e colocar o seguinte código:

```
const express = require('express');
const app = express();

// GET simples
app.get('/', (req, res) => {
  res.send('Bem-vindo ao Express com Node.js!');
});

// Inicializar servidor
app.listen(3000, () => {
  console.log('Servidor disponível em http://localhost:3000');
});
```

1 2 3 4

Implementação de uma REST API com Nodejs e Express

1) Importa a biblioteca: express

2) Cria uma instância de uma aplicação express

3) Define uma rota GET para o caminho '/' (a página inicial). O método app.get() recebe dois parâmetros:

- O caminho da rota ('/').
- Uma função callback: Essa função recebe dois objetos:
  - req (request): contém informações sobre o pedido do cliente
  - res (response): usado para enviar a resposta de volta ao cliente.
- res.send() envia uma resposta de texto com a mensagem:"Bem-vindo ao Express com Node.js!

4) Inicia o servidor na porta 3000 e apresenta uma mensagem no terminal.

Event loop

O Event Loop é o "coração" do Node.js. É um mecanismo que permite ao Node realizar várias tarefas ao mesmo tempo (como lidar com milhares de pedidos num servidor) usando, na maior parte do tempo, apenas uma thread principal.

É um cérebro que gere uma lista de tarefas, garantindo que a thread principal nunca fica "bloqueada" à espera de nada.

Event loop

Imaginemos o Node.js como um restaurante com um único empregado de mesa (esta é a *thread principal*).

Este empregado é super rápido, mas só consegue fazer uma coisa de *cada vez*. O **Event Loop** é o *sistema de gestão de pedidos* deste empregado.

Event loop - Funcionamento

1. Pedidos Rápidos (Síncronos): Se um cliente pede um copo de água (uma tarefa rápida, como 2+2), o empregado serve-o *imediatamente* e no local. (Isto acontece na Call Stack).
2. Pedidos Lentos (Assíncronos): Se um cliente pede um assado (uma tarefa lenta, como ler um ficheiro grande ou pedir dados a outra API), o empregado não fica à espera ao lado da mesa.
3. Delegar: O empregado anota o pedido, entrega-o à cozinha (as APIs internas do Node ou o sistema operativo) e diz: "Avisem-me quando estiver pronto!"
4. Continuar a Trabalhar: O empregado volta imediatamente para atender outros clientes (processar outras tarefas rápidas). A thread principal fica livre.

Event loop - Funcionamento

5. A "Sineta" (Callback): Quando a cozinha termina o assado (a tarefa assíncrona termina), toca uma "sineta".

6. A Fila (Callback Queue): O Event Loop ouve a sineta e coloca uma nota na "fila de entrega" (a Callback Queue): "Entregar assado ao cliente X".

7. O "Loop" (O Ciclo): Assim que o empregado termina as tarefas rápidas que estava a fazer (a *Call Stack* fica vazia), ele pergunta ao Event Loop: "O que faço a seguir?" O Loop verifica a fila e diz: "Leva este assado!"

Implementação de uma REST API com Nodejs e Express

- Etapas para testar o exemplo:
  - npm init –y : inicializa a aplicação com o ficheiro de configuração package.json
  - npm install express : instala a biblioteca express
  - Criar um ficheiros app.js com o código de exemplo fornecido
  - Executar no terminal: node app.js
  - Abrir o browser e digitar: http://localhost:3000

Exemplo Bookstore com MongoDB

```
const express = require('express');
const { MongoClient } = require('mongodb');

const app = express();
const uri = 'connectionString';
const client = new MongoClient(uri);

let db; // variável global para guardar a referência à base de dados

async function connectToMongo() {
  await client.connect();
  db = client.db('bookstore');
  console.log('Ligação ao MongoDB estabelecida.');
}
(...)
```

1

2

3

snappify.com

Exemplo Bookstore com MongoDB

1) cria um objeto cliente MongoDB usando a URI de conexão (devolve um objeto, por isso temos de utilizar {}

2) declara uma variável que pode ser alterada

3) async: indica que a função é assíncrona (pode demorar tempo a executar)

   await: espera que a conexão termine antes de continuar

   client.db**('bookstore')**: acede à base de dados chamada 'bookstore'

   console.log(): imprime uma mensagem no terminal

Recurso Books

```
(...)
app.get('/books', async (req, res) => {
  try {
    const books = await db.collection('books').find({}).toArray();
    res.status(200).json(books);
  } catch (err) {
    console.error('Erro ao obter livros:', err);
    res.status(500).json({ error: 'Falha ao obter livros' });
  }
});

connectToMongo().then(() => {
  app.listen(3000, () => console.log('Servidor disponível em http://localhost:3000'));
});
```

4

5

Exemplo Bookstore com MongoDB

4) **app.get('/books', ...)**: define uma rota que responde a pedidos GET no endereço /books

req, res: objetos que representam o pedido (request) e a resposta (response)

try...catch: estrutura para lidar com erros

.find({}): procura todos os documentos na coleção 'books'

.toArray(): converte os resultados em array

res.status(200): código HTTP 200 significa sucesso

.json(): envia a resposta em formato JSON

catch (err): captura erros se algo correr mal

status(500): código HTTP 500 indica erro no servidor

Mais informação sobre status codes

Exemplo Bookstore com MongoDB

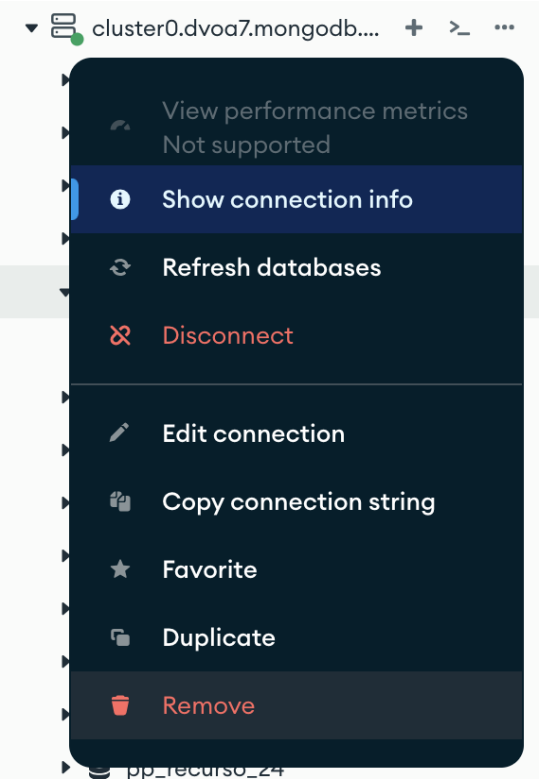
5) **.then()**: executa o código depois da conexão estar completa

app.listen(3000): coloca o servidor à escuta na porta 3000

Callback function: função que é executada quando o servidor arranca

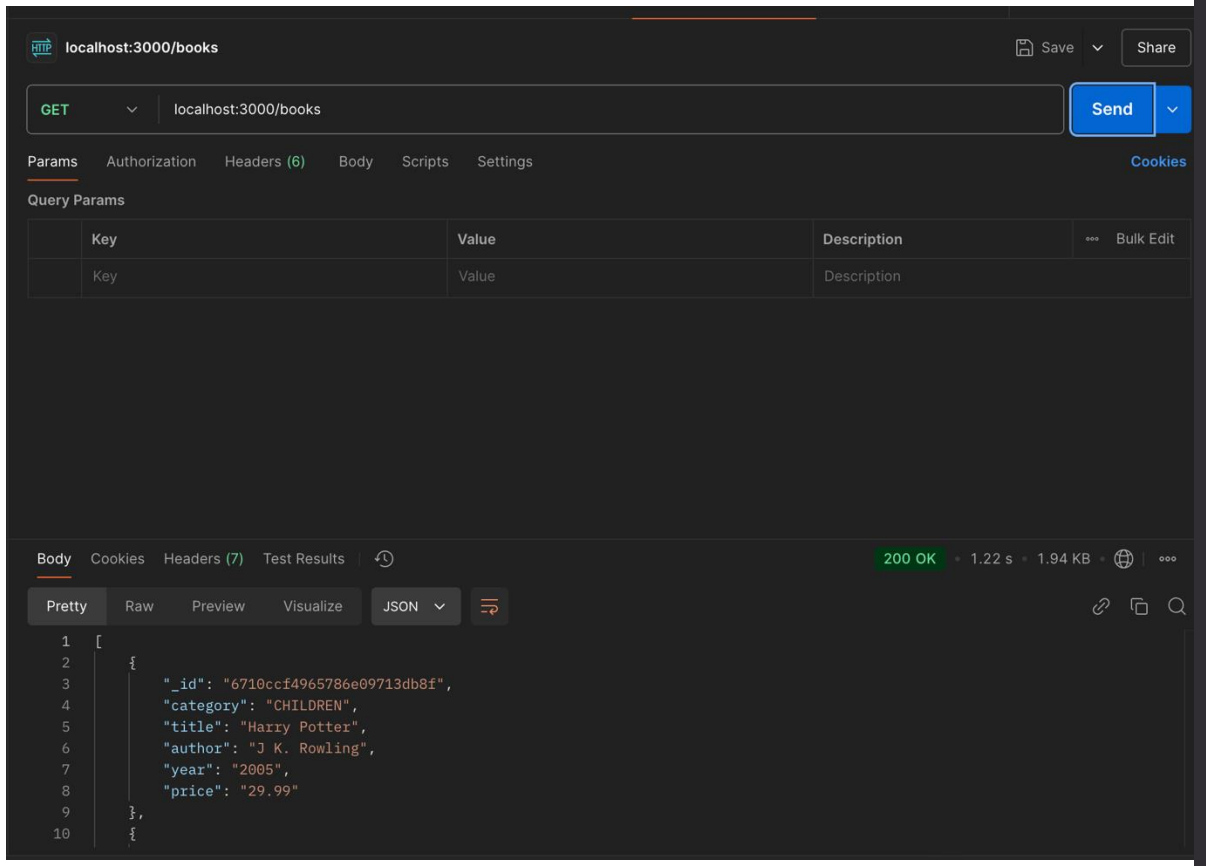
Recurso Books

- A connection string do mongo pode ser obtida no Mongo Compass:



Recurso Books

- Testar utilizando (por exemplo) Postman:



Recurso Book com id (mongoID)

Alterar:

```
const { MongoClient, ObjectId } = require('mongodb');
```

```
app.get('/books/:idLivro', async (req, res) => {
  try {
    const { idLivro } = req.params; 1
    const book = await db.collection('books').findOne({ _id: new ObjectId(idLivro) });

    if (!book) {
      return res.status(404).json({ error: 'Livro não encontrado' });
    }

    res.status(200).json(book);
  } catch (err) {
    console.error('Erro ao obter livro:', err);
    res.status(500).json({ error: 'Falha ao obter livro' });
  }
});
```

Recurso Book com id (mongoID)

1) Quando alguém acede a http://localhost:3000/books/507f1f77bcf86cd799439011, o Express captura 507f1f77bcf86cd799439011 e coloca-o num objeto:

```
req.params = {
  idLivro: "507f1f77bcf86cd799439011"
}
```

O nome idLivro tem que **corresponder** ao nome na rota.

Recurso Book com id (mongoID)

GET localhost:3000/books/6710ccf4965786e09713db93 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit

Body Cookies Headers (7) Test Results Status: 200 OK Time: 123 ms Size: 386 B Save Response

Pretty Raw Preview Visualize JSON


```
{
    "_id": "6710ccf4965786e09713db93",
    "category": "ROMANCE",
    "title": "XML In A Nutshell",
    "authors": {
        "author": "W. Scott Means"
    },
    "year": "2001",
    "price": "9.95"
}
```

Recurso book com consulta livre

```javascript
// rota para consulta livre com query parameter
app.get('/books/search', async (req, res) => {
    try {
        // extrai o parâmetro 'free-query' da URL e renomeia para 'freeQuery'
        // destructuring com renomeação: { 'nome-da-URL': nomeVariavel }
        const { 'free-query': freeQuery } = req.query;

        if (!freeQuery) {
            return res.status(400).json({ error: 'Parâmetro free-query é obrigatório' });
        }

        // cria um filtro de pesquisa em múltiplos campos
        const filter = {
            $or: [
                { title: { $regex: freeQuery, $options: 'i' } },
                { author: { $regex: freeQuery, $options: 'i' } },
                { genre: { $regex: freeQuery, $options: 'i' } }
            ]
        };

        const books = await db.collection('books').find(filter).toArray();

        if (books.length === 0) {
            return res.status(404).json({ error: 'Nenhum livro encontrado' });
        }

        res.status(200).json(books);
    } catch (err) {
        console.error('Erro ao pesquisar livros:', err);
        res.status(500).json({ error: 'Falha ao pesquisar livros' });
    }
});
```

snappify.com

Recurso book com consulta livre

GET http://localhost:3000/books/search?free-query=rowling Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

☑	free-query	rowling
	Key	Value

Body Cookies Headers (7) Test Results Status: 200 OK Time: 104 ms Size: 371 B Save Response

Pretty Raw Preview Visualize JSON


```
[
    {
        "_id": "6710ccf4965786e09713db8f",
        "category": "CHILDREN",
        "title": "Harry Potter",
        "author": "J K. Rowling",
        "year": "2005",
        "price": "29.99"
    }
```

Recurso add book

Adicionar:
const app = express();
app.use(express.json()); // middleware para processar JSON no body

```
app.post('/books', async (req, res) => {
  try {
    const newBook = req.body;

    // validação básica
    if (!newBook.title || !newBook.author) {
      return res.status(400).json({ error: 'Título e autor são obrigatórios' });
    }

    const result = await db.collection('books').insertOne(newBook);
    const insertedBook = await db.collection('books').findOne({ _id: result.insertedId });

    res.status(201).json(insertedBook);
  } catch (err) {
    console.error('Erro ao adicionar livro:', err);
    res.status(500).json({ error: 'Falha ao adicionar livro' });
  }
});
```

Recurso add book

- app.use(express.json());
  - **Middleware** = "meio do caminho" = função que está **entre** o pedido (request) e a resposta (response).
  - O middleware:
    - Processa ou modifica o pedido (req) antes de chegar à rota
    - Processa ou modifica a resposta (res)
    - Executa código
    - Termina o ciclo pedido-resposta
    - Chama o próximo middleware na cadeia
  - Neste caso em particular:
    - Intercepta todos os pedidos que chegam ao servidor
    - Verifica se o body contém JSON
    - Converte o JSON em objeto JavaScript
    - Adiciona esse objeto a req.body
    - Passa o controlo para o próximo middleware ou rota

Recurso add book

POST localhost:3000/book Send

Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2     "category": "Fantasy",
3     "title": "Game of Thrones",
4     "author": "George R. R. Martin",
5     "year": "2015",
6     "price": "30.00"
7 }
8
```

Body Cookies Headers (7) Test Results 201 Created • 1.26 s • 383 B

Pretty Raw Preview Visualize JSON

```
1 {
2     "category": "Fantasy",
3     "title": "Game of Thrones",
4     "author": "George R. R. Martin",
5     "year": "2015",
6     "price": "30.00",
7     "_id": "6759d1e6776b940ee22f74e9"
8 }
```

Recurso update book

```
app.patch('/books/:idLivro', async (req, res) => {
  try {
    const { idLivro } = req.params;
    const updates = req.body;

    // remove o _id se vier no body (não pode ser atualizado)
    delete updates._id;

    // verifica se há campos para atualizar
    if (Object.keys(updates).length === 0) {
      return res.status(400).json({ error: 'Nenhum campo para atualizar' });
    }

    const result = await db.collection('books').updateOne(
      { _id: new ObjectId(idLivro) },
      { $set: updates }
    );

    if (result.matchedCount === 0) {
      return res.status(404).json({ error: 'Livro não encontrado' });
    }

    res.status(200).json({ message: 'Livro atualizado com sucesso' });
  } catch (err) {
    console.error('Erro ao atualizar livro:', err);
    res.status(500).json({ error: 'Falha ao atualizar livro' });
  }
});
```

snappify.com

Recurso update book

PATCH localhost:3000/books/6710ccf4965786e09713db93

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "price": 15.99,
3   "year": 1950
4 }
```

Body Cookies Headers (7) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Livro atualizado com sucesso"
3 }
```

Recurso replace book

```
app.put('/books/:idLivro', async (req, res) => {
    try {
        const { idLivro } = req.params;
        const newBook = req.body;

        // remove o _id se vier no body (não pode ser atualizado)
        delete newBook._id;

        // validação básica
        if (!newBook.title || !newBook.author) {
            return res.status(400).json({ error: 'Título e autor são obrigatórios' });
        }

        const result = await db.collection('books').replaceOne(
            { _id: new ObjectId(idLivro) },
            newBook
        );

        if (result.matchedCount === 0) {
            return res.status(404).json({ error: 'Livro não encontrado' });
        }

        res.status(200).json({ message: 'Livro substituído com sucesso' });
    } catch (err) {
        console.error('Erro ao substituir livro:', err);
        res.status(500).json({ error: 'Falha ao substituir livro' });
    }
});
```

snappify.com

Recurso replace book

PUT localhost:3000/books/6710ccf4965786e09713db93 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
  "_id": {
    "$oid": "6710ccf4965786e09713db93"
  },
  "category": "TECH",
  "title": "XML In A Nutshell",
  "author": "W. Scott Means",
  "year": 1951,
  "price": 15.99
}
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 167 ms Size: 279 B Save Response

Pretty Raw Preview Visualize JSON

```
{
  "message": "Livro substituído com sucesso"
}
```

Recurso delete book

```
app.delete('/books/:idLivro', async (req, res) => {
    try {
        const { idLivro } = req.params;

        const result = await db.collection('books').deleteOne(
            { _id: new ObjectId(idLivro) }
        );

        if (result.deletedCount === 0) {
            return res.status(404).json({ error: 'Livro não encontrado' });
        }

        res.status(200).json({ message: 'Livro removido com sucesso' });
    } catch (err) {
        console.error('Erro ao remover livro:', err);
        res.status(500).json({ error: 'Falha ao remover livro' });
    }
});
```

snappify.com

Referências

- Referências Web:
  - https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Basico_sobre_HTTP
  - https://www.restapitutorial.com/
  - https://www.postman.com
  - https://www.mongodb.com/resources/languages/express-mongodb-rest-api-tutorial
- Livro:
  - Lauret A., The Design of Web APIs, Manning, 2019;

P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Introdução Web APIs

Bruno Oliveira (bmo@estg.ipp.pt)

PEI Processamento Estruturado de Informação